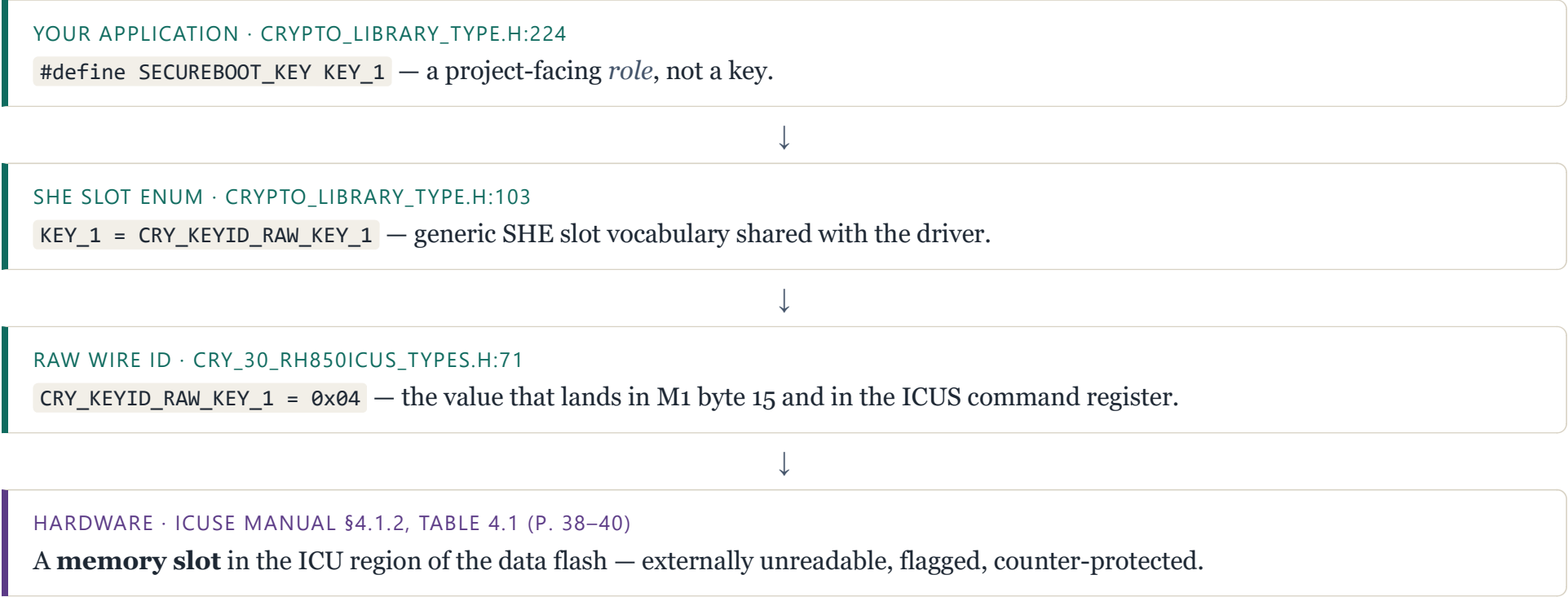


Where do `SECUREBOOT_KEY` and `DATASTORE_KEY` actually live?

The one-sentence answer

They don't exist in the hardware at all — they are **compile-time aliases** defined in `crypto_library_type.h` that resolve to generic SHE slots (`KEY_1 ... KEY_4`), which the Vector driver provisions into ICUSE memory slots in the data flash via the standard SHE $M1|M2|M3$ key-update protocol.

1 · Follow one name through four layers



By default: `SECUREBOOT_KEY`→`KEY_1` , `JTAG_KEY`→`KEY_2` , `DATASTORE_KEY`→`KEY_3` , `INTEGRITY_KEY`→`KEY_4` . If `KEYRAM_AS_KEY` is defined, all four collapse onto the volatile `RAM_KEY` instead [`crypto_library_type.h:214–228`].

2 · How the key value gets into the slot

There is no factory-injected key here and no key arrives over a bus. `SecSB_EnableSecurity` (`secure_boot.c:127`) manufactures each key on-target:

Step 1 — generate. The ICUSE TRNG produces 16 random bytes; they are staged into the volatile `RAM_KEY` as a plaintext key
[`SecSB_GenerateRamKey` → `SecLib_KeyRAMUpdate` , `crypto_library.c:397`; manual §4.1.3].

Step 2 — wrap in the SHE update protocol. `SecLib_GenerateM1M2M3` (`key_manager.c:99`) builds the three-part message of manual §4.4: $M1 = UID | ID | AuthID$, $M2$ = the new key value, its `KEY_FLAG` (`KU|WP`) and counter=1, AES-CBC-encrypted under $K1$; $M3$ = a CMAC over $M1|M2$ under $K2$. $K1$ – $K2$ are derived from the authenticating key's material via the SHE KDF [manual §4.4, p. 44].

Step 3 — deliver to hardware. `SecLib_KeySHEUpdate` (`crypto_library.c:456`) hands the 64-byte $M1M2M3$ to `Cry_30_Rh850Icus_KeyExtract` , which parses ID/AuthID out of $M1$ byte 15 and finally issues `CMD_LOAD_KEY` with the slot number [`Cry_30_Rh850Icus_Hw.c:1100`; manual §5.6.7]. Inside ICUSE, the message is decrypted and verified, and the slot is written with key + flags + counter.

Step 4 — prove it. `SecSB_CreateKey` immediately runs a CMAC generate-and-verify round-trip with the new slot (`secure_boot.c:463–480`). Success = the slot is live. Because the flags included **WP**, it is now frozen — which is why re-running enable-security later surfaces `E_VC_SECBOOT_ICUS_KEY_WRITE_PROTECTED` and the code treats that as "already enabled" rather than an error (`secure_boot.c:142`).

THE CHICKEN-AND-EGG, RESOLVED

How can $M3$ be authenticated by a key that doesn't exist yet? Because the project loads each slot with `authKeyId = keyId` and an all-zero auth key. The manual sanctions exactly this: an unregistered AuthID equal to ID makes `KEY_AuthID` behave as `0...0 128` [§4.4, p. 43]. Everything is self-authenticated on first load and TRNG-born — nothing secret ever crosses the wire in plaintext except into `RAM_KEY`.

3 · Knowledge check

Q1 · NVM key slots physically reside in which area?

code flash boot region

data flash ICU region

internal RAM key store

external SPI flash memory

Manual §4.1.2: ICUSE uses part of the **data flash** (the ICU region) for key memory slots; the RAM key is the only volatile one.

Q2 · Which raw KEY_ID does SECUREBOOT_KEY carry on the wire by default?

raw key identifier 0x04

raw key identifier 0x01

raw key identifier 0x0E

raw key identifier 0x06

SECUREBOOT_KEY → KEY_1 → CRY_KEYID_RAW_KEY_1 = 0x04 (crypto_library_type.h:224, Cry_30_Rh850Icus_Types.h:71).
0x01 is MASTER_ECU_KEY, 0x0E is RAM_KEY, 0x06 is KEY_3 (DATASTORE_KEY).

Q3 · Which KEY_FLAG combination is applied when provisioning these keys?

key usage and boot protection

boot protection and debugger protection

key usage and write protection

disable wildcard and verify only

secure_boot.c:365,442 — keyFlags = KU | WP : CMAC-capable and permanently frozen after the first load.

Q4 · What protects the key value travelling inside message M2?

CMAC with derived key K2

AES ECB with derived key K3

AES ECB with derived key K1

AES CBC with derived key K1

M2 = ENC_{CBC,K1,IV=0}(Counter | KEY_FLAG | zeros | KEYID) — manual §4.4, p. 44. K2's CMAC covers M3, K3's ECB encrypts M4.

4 · Read next / practice

PRIMARY SOURCE

ICUSE User Manual (docs/rh850_f1km_icuse_usermanual.pdf), **Section 4.4 "How to Register Keys", pages 43–45** — the normative description of M1–M5, the KDF constants, and which AuthID may register which key. Read it once with the key-slot map open.

- Keep the Key Slot Map & Glossary open as you re-read SecSB_EnableSecurity top-to-bottom; every name on the map appears in that function.
- Then trace, on paper: what KEY_ID, flags and counter SecSB_CreateKey(DATASTORE_KEY, CMAC_KEY) puts into its M1M2M3.

YOUR TEACHER

Anything unclear — the M1/M2 bit layout, the KDF, why WP is safe, the `KEYRAM_AS_KEY` trade-off — just ask in the chat. Follow-up questions are the fastest way to lock this in.